LAB SHEET 9

Deploying Custom Socket Applications on Mininet-WiFi Nodes

Week	9
Duration	2 hours
Deliverable	echo_bench_client.py, app_topo.py, rtt_results.json, rtt_plot.png, report (PDF), pushed to your netprog-2026 GitHub repository

Learning Objectives

By the end of this lab, you should be able to:

1. Explain why a custom Python socket application written for a single machine can run unmodified inside an emulated Mininet-WiFi node.

2. Launch a custom server application on one station and a client application on another using station.cmd() from a Mininet-WiFi script.

3. Write a non-interactive benchmark client that generates sustained application traffic and times each message.

4. Measure application-level round-trip time under different wireless link conditions and compare it to Lab 8's iperf3-based measurements.

5. Visualize application-level performance data using matplotlib, extending Lab 8's plotting workflow

Background

Every Mininet-WiFi station is a genuine Linux network namespace with its own IP stack — this is the emulation principle that has run through this course since Lab 1. A direct consequence is that any ordinary Python program using the socket module, including the TCP echo server you wrote in Lab 2, needs no code changes at all to run inside a station: from the program's point of view, it is simply running on "a machine" with a network interface,

node.cmd(command)	Runs a shell command inside the node and blocks until it finishes, returning its output as a string. Used for short commands or backgrounding a server with a trailing &.
node.popen(command)	Runs a command inside the node and returns a Python subprocess handle you can read from, write to, or terminate while the topology keeps running.
node.IP()	Returns the node's IP address as a string — exactly what you need to tell a client where to connect.

Today's client, however, cannot be Lab 2's original interactive client.py — it used input() to read a message from a human at a keyboard, which has no meaning inside an unattended script launched via cmd(). You will write a new, non-interactive benchmark client that automatically sends a fixed number of messages and times each one, so the whole experiment can run start-to-finish from a single Python script with no manual typing.

> **Zero code changes, one new client**
>
> echo_server.py from Lab 2 is deployed today completely unmodified — copy it into today's folder and it just works. Only the client needs to change, and only because it was interactive, not because anything about sockets or Mininet-WiFi is different.

Prerequisites

- Lab 2 completed: a working, threaded echo_server.py.
- Lab 6 completed (addStation(), addAccessPoint(), setPosition()).
- Lab 8 completed (comfort with node.cmd()-driven measurement, JSON result files, and matplotlib plotting with the Agg backend).

Software & Materials

- Mininet-WiFi (installed in Lab 1), Python 3.8+
- Your echo_server.py from Lab 2 (final, threaded version)
- matplotlib

Lab Tasks

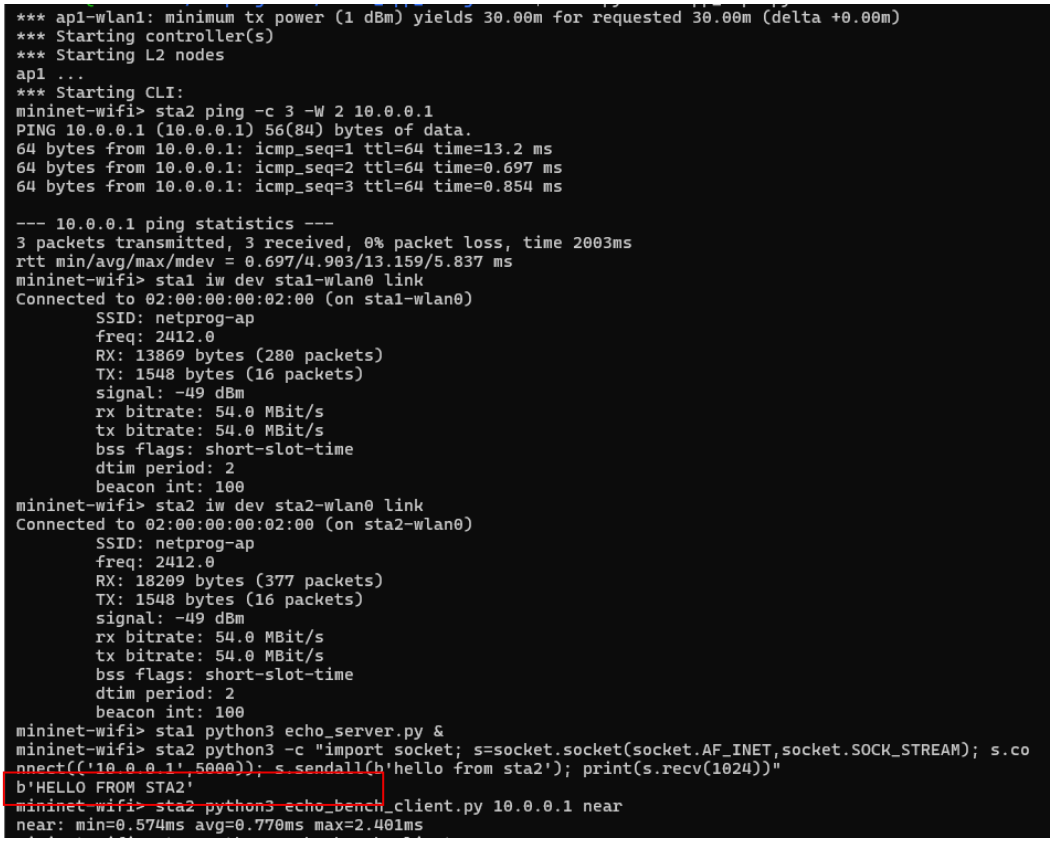
Complete the following tasks and record evidence (terminal output / screenshots) for each in your report.

Task 1: Screenshot of Part B's manual test, showing echo_server.py (unmodified from Lab 2) responding correctly inside sta1.

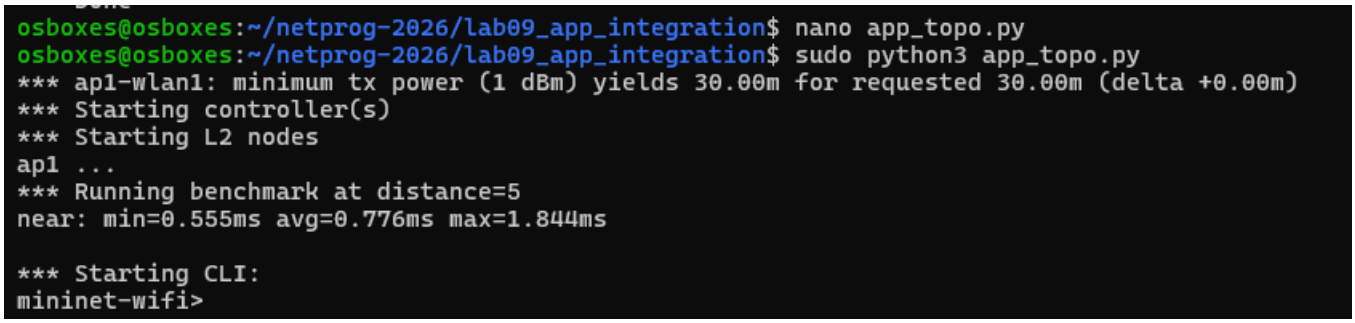
```
osboxes@osboxes:~/netprog-2026/lab09_app_integration$ sudo python3 app_topo.py
*** ap1-wlan1: minimum tx power (1 dBm) yields 30.00m for requested 30.00m (delta +0.00m)
*** Starting controller(s)
*** Starting L2 nodes
ap1 ...
*** Starting CLI:
mininet-wifi> sta2 ping -c 3 -W 2 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=13.2 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.697 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.854 ms

--- 10.0.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.697/4.903/13.159/5.837 ms
mininet-wifi> sta1 iw dev sta1-wlan0 link
Connected to 02:00:00:00:02:00 (on sta1-wlan0)
        SSID: netprog-ap
        freq: 2412.0
        RX: 13869 bytes (280 packets)
        TX: 1548 bytes (16 packets)
        signal: -49 dBm
        rx bitrate: 54.0 MBit/s
        tx bitrate: 54.0 MBit/s
        bss flags: short-slot-time
        dtim period: 2
        beacon int: 100
mininet-wifi> sta2 iw dev sta2-wlan0 link
Connected to 02:00:00:00:02:00 (on sta2-wlan0)
        SSID: netprog-ap
        freq: 2412.0
        RX: 18209 bytes (377 packets)
        TX: 1548 bytes (16 packets)
        signal: -49 dBm
        rx bitrate: 54.0 MBit/s
        tx bitrate: 54.0 MBit/s
        bss flags: short-slot-time
        dtim period: 2
        beacon int: 100
mininet-wifi> sta1 python3 echo_server.py &
mininet-wifi> sta2 python3 -c "import socket; s=socket.socket(socket.AF_INET,socket.SOCK_STREAM); s.co
nnect(('10.0.0.1',5000)); s.sendall(b'hello from sta2'); print(s.recv(1024))"
b'HELLO FROM STA2'
mininet-wifi>
```

Task 2: Screenshot of echo_bench_client.py's summary line (min/avg/max) from a single run.



Task 3: Screenshot of Part D's full automated run (topology build through benchmark summary) in one terminal session.



Task 4: Screenshots of both Part E summary lines (near and far), with the far-distance run showing a visibly higher average and/or more variable RTT.

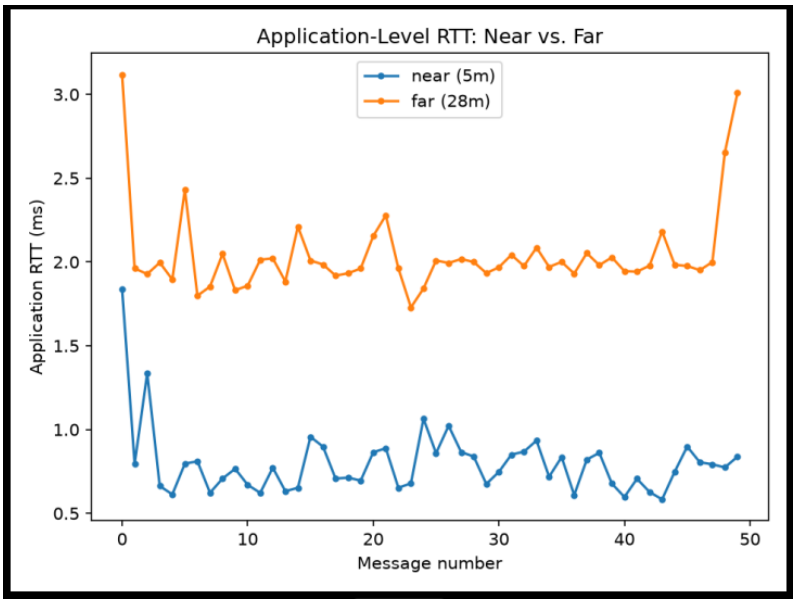
```
osboxes@osboxes:~/netprog-2026/lab09_app_integration$ sudo python3 app_topo.py
*** ap1-wlan1: minimum tx power (1 dBm) yields 30.00m for requested 30.00m (delta +0.00m)
*** Starting controller(s)
*** Starting L2 nodes
ap1 ...
*** Running benchmark at distance=5
near: min=0.583ms avg=0.799ms max=1.838ms

*** Running benchmark at distance=28
far: min=1.729ms avg=2.045ms max=3.118ms

*** Starting CLI:
mininet-wifi>
```

Task 5: Include rtt_plot.png (or a screenshot of it) and describe the visible difference between the near and far series.

```
osboxes@osboxes:~/netprog-2026/lab09_app_integration$ nano plot_rtt.py
osboxes@osboxes:~/netprog-2026/lab09_app_integration$ source ~/netprog-env/bin/activate
(netprog-env) osboxes@osboxes:~/netprog-2026/lab09_app_integration$ python3 plot_rtt.py
Saved rtt_plot.png
(netprog-env) osboxes@osboxes:~/netprog-2026/lab09_app_integration$ xdg-open rtt_plot.png
(netprog-env) osboxes@osboxes:~/netprog-2026/lab09_app_integration$ deactivate
```



A comparison of the Round-Trip Time (RTT) for 50 messages at distances of 5 meters and 28 meters reveals that, at 5 meters, the RTT generally falls within the 0.6–1.0 ms range. In contrast, at 28 meters, the RTT is typically between 1.8–2.2 ms, occasionally reaching as high as 3.1 ms. Throughout the experiment, the line representing the greater distance ("far") consistently remains above the line for the shorter distance ("near"), demonstrating that, under the tested conditions, a station located further from the Access Point (AP) clearly experiences a longer delay in receiving a response.

Task 6: In 2-3 sentences, reflect on the fact that echo_server.py needed zero code changes to run inside Mininet-WiFi, and connect this to the emulation-vs-simulation theme from Lab 1; also note one difference between this lab's application-level RTT and Lab 8's iperf3-based measurements at similar distances.

Answer You can use `echo_server.py` from Lab 2 without any code modifications because the stations in Mininet-WiFi utilize a real Linux network stack and support the standard socket API. This demonstrates the concept of emulation, where actual programs run on a simulated network distinct from simulation, which might rely on behavioral models rather than the actual software. While Lab 8 uses `iperf3` (via UDP) to measure throughput, jitter, and packet loss, this lab measures application-level RTT using TCP, which accounts for both the program's processing time and the effects of TCP's behavior.

Verification Checklist

#	Checkpoint	From	✓
1	Unmodified echo_server.py runs inside sta1 and responds correctly (Part B)	Task 1	☑
2	echo_bench_client.py connects once and correctly times count messages (Part C)	Task 2	☑
3	app_topo.py automates server launch + one benchmark run end-to-end (Part D)	Task 3	☑
4	Near and far benchmark runs both complete and produce rtt_near.json / rtt_far.json (Part E)	Task 4	☑
5	rtt_plot.png shows both series clearly labeled and distinguishable (Part F)	Task 5	☑
6	Far-distance RTT is visibly higher/more variable than near-distance RTT	Task 5 (Graph image)	☑

Reflection Questions

Answer briefly (2-4 sentences each) in your report:

1.Explain why echo_server.py from Lab 2 could be deployed inside a Mininet-WiFi station without any code modification. What does this tell you about the relationship between emulation and ordinary application code?

Answer Stations in Mininet-WiFi possess their own Linux network stack and IP addresses, allowing programs to utilize the socket API just as they would when running on a standard Linux machine. This demonstrates that emulation can execute actual application code, in contrast to simulation, which might rely solely on behavioral models of applications.

2. Why was a new, non-interactive benchmark client needed instead of directly reusing Lab 2's interactive client.py for this lab's automated measurements?

Answer The original client used `input()` to wait for user text entry, which is unsuitable for automated testing. The new client sends a predefined number of messages and tracks the timing for each round, making it easier to perform repetitions and compare results.

3. Compare your application-level RTT measurements (Part E) to Lab 8's iperf3-based measurements at similar distances. What might explain any differences you observed between the two measurement approaches?

Answer Lab 8 measures data reception rate, jitter, and packet loss for UDP, whereas this lab measures the round-trip time (RTT) of application-level TCP messages; therefore, the values cannot be directly compared. Furthermore, the RTT is influenced by factors such as server processing time, operating system scheduling, and TCP retransmissions. Consequently, the RTT may fluctuate even if the throughput in Lab 8 remains constant and UDP packet loss is zero.

End of Lab Sheet 9